

Revised Report - What Happens Through PredictionService

PredictionService is the online industrial ML serving layer of the MDF1 ML platform. Its responsibility is to consume the production-ready artifacts created by TrainingService and serve them safely through APIs for prediction, explainability, recommendations, database traceability, and GUI/MES integration.

The service was refactored toward registry-driven serving, centralized settings, artifact synchronization, exact feature-order enforcement, weighted ensemble support, modular utilities, decorator-based logging, SHAP explainability, and industrial-safe runtime validation.

High-Level Runtime Flow

```
TrainingService publishes artifacts
↓
best_models_registry.json + model bundles
↓
PredictionService startup
↓
Artifact synchronization
↓
ModelRegistry loads registry and artifact bundles
↓
API receives prediction request
↓
InputValidator checks feature contract
↓
Predictor executes single model or weighted ensemble
↓
Explainer generates SHAP explanation
↓
RiskLevel converts prediction into LOW / MEDIUM / HIGH
↓
RecommendationEngine builds operator guidance
↓
PredictionLogRepository saves traceability record
↓
GUIService / Swagger / future MES consumes the response
```

1. Role of PredictionService

TrainingService is offline and experimental by nature: it cleans data, engineers features, trains models, evaluates them, and publishes artifacts. PredictionService is online and operational: it must be stable, reproducible, traceable, and safe.

The main principle is: TrainingService decides; PredictionService enforces and serves. This prevents PredictionService from becoming another training pipeline or from inventing its own feature/preprocessing behavior.

2. Central Settings Architecture

The config/settings.py file became the central location for service configuration. It defines paths, registry files, artifact contract names, database URL, logging paths, feature governance rules, ensemble flags, SHAP settings, and TrainingService contract flags.

Why this matters: without centralized settings, paths and behavior become scattered across API, service, and utility files. That makes deployment risky and hard to debug. Centralized settings allows Docker, local development, and future server deployment to use the same logic with environment variables.

```
MODEL_FILE_NAME = "model.joblib"
FEATURES_FILE_NAME = "features.json"
METRICS_FILE_NAME = "metrics.json"
METADATA_FILE_NAME = "metadata.json"
REGISTRY_FILE = ARTIFACTS_DIR / "best_models_registry.json"
```

The `REQUIRED_ARTIFACT_FILES` list is now dynamically created from requirement flags. This means the artifact contract can evolve safely. For example, `metrics.json` may remain required now, while future optional SHAP files can be added later without breaking the core serving contract.

3. Utilities and Decorators

The utils layer contains reusable technical helpers that are not domain-specific business logic.

File / Class / Function	What it does	Why it exists
<code>logging_utils.get_logger</code>	Creates a consistent logger with console/file formatting.	Prevents mixed logging styles and supports traceability.
<code>decorators.service_step</code>	Combines execution logging, latency measurement, and error logging.	Avoids repeating try/log/timing logic in every function.
<code>time_utils.TimeBudget</code>	Context manager for measuring execution duration.	Useful for loading artifacts, predictions, SHAP, and DB calls.
<code>artifact_sync.sync_best_artifacts</code>	Copies registry-selected artifacts from <code>TrainingService</code> to <code>PredictionService</code> .	Keeps serving artifacts aligned with training output.
<code>sync_artifacts.main</code>	Manual script entry point for artifact synchronization.	Useful for local testing and controlled refresh.

4. Core Model Registry

`core/model_registry.py` is the most important infrastructure file in `PredictionService`. It replaced duplicated `service/model_registry` logic and became the single source of truth for registry and artifact loading.

Class / Method	Functionality	Method / Algorithm Used
<code>LoadedArtifact</code>	Dataclass holding one loaded model artifact, feature list, metrics, metadata, and load timestamp.	Artifact bundle representation.
<code>RegistryEntry</code>	Dataclass representing one target entry from <code>best_models_registry.json</code> .	Registry-driven serving contract.
<code>ModelRegistry.load_registry</code>	Reads and validates <code>best_models_registry.json</code> .	JSON parsing and serving-type validation.
<code>ModelRegistry.load_all_from_registry</code>	Loads only models referenced by the registry.	Collects required artifacts from <code>single_model</code> and <code>weighted_ensemble</code> entries.
<code>ModelRegistry.load_one</code>	Loads <code>model.joblib</code> , <code>features.json</code> , <code>metrics.json</code> , <code>metadata.json</code> .	<code>joblib.load</code> for sklearn pipeline; JSON loading for contract files.
<code>get_artifact_for_target</code>	Returns the artifact for a single-model target.	Target-to-artifact resolution.
<code>get_ensemble_members_for_target</code>	Returns member key, weight, and loaded artifact for ensemble serving.	Weighted ensemble orchestration.

Why this matters: `PredictionService` should not scan random folders and guess what to serve. It should serve only what `TrainingService` selected and registered. This supports governance, rollback, ensemble deployment, and safer production behavior.

5. Input Validation and Feature Contract Enforcement

`service/input_validator.py` protects the model from unsafe runtime inputs. It compares the incoming payload with the exact saved feature list from `features.json`.

Function	What it does	Why it matters
analyze_input_features	Detects missing, extra, invalid numeric, null, and forced-missing features.	Provides diagnostics before prediction.
validate_input_features	Raises errors depending on settings flags.	Prevents hidden corrupted predictions.
build_ordered_feature_frame	Creates a pandas DataFrame using the exact expected feature order.	Prevents feature-order mismatch, one of the most dangerous ML serving errors.

Specific method used: numeric coercion converts incoming values to float when enabled. NaN and infinity are rejected when the NaN guard is active. The final DataFrame is explicitly reordered by features.json.

6. Prediction Logic

service/predictor.py performs the actual model inference. It does not load files itself; it receives LoadedArtifact objects from ModelRegistry. This separation keeps prediction logic clean and testable.

Function	Functionality	Algorithm / Method
_extract_prediction_value	Converts sklearn prediction output into a float.	Handles array-like and scalar outputs.
_get_algorithm_name	Reads algorithm name from metadata.json.	Metadata-driven algorithm reporting.
_predict_from_artifact	Validates input, builds ordered frame, calls artifact.predictor.predict().	sklearn Pipeline prediction.
predict_single_model	Returns prediction and diagnostics for one artifact.	Single-model inference.
predict_weighted_ensemble	Runs each ensemble member and combines predictions.	Weighted average ensemble.

```
Weighted ensemble formula:
final_prediction = sum(weight_i * prediction_i) / sum(weight_i)
```

Why weighted ensemble is useful: XGBoost may capture strong nonlinear interactions, while Random Forest may provide stable behavior on noisy industrial data. Combining them can improve robustness when TrainingService proves the ensemble improves validation performance.

7. Explainability Layer

service/explainer.py and service/explanation_service.py provide local runtime explainability. PredictionService uses SHAP - SHapley Additive exPlanations - to estimate how much each feature contributed to a specific prediction.

File / Function	Functionality	Method / Algorithm
explanation_service.explain_prediction	Coordinates prediction, SHAP explanation, risk, and recommendations.	Service orchestration.
_select_explanation_artifact	For ensembles, selects an explainable member, preferably XGB.	Explainability member selection.
explainer._extract_pipeline_components	Gets imputer and model from sklearn Pipeline.	Pipeline component extraction.
explainer.explain_single_model_prediction	Transforms input, runs TreeExplainer, ranks impacts.	SHAP TreeExplainer.

SHAP output includes feature, value, impact, absolute impact, direction, and watched-feature flag. Positive impact means the feature pushed the prediction upward; negative impact means it pushed the prediction downward.

8. Recommendation Engine

service/recommendation_engine.py converts SHAP impacts into operator-friendly language. Raw SHAP values are useful for engineers but difficult for operators, so the recommendation layer translates model explanation into process-oriented messages.

It uses FEATURE_LABELS and WATCH_FEATURES from settings.py. This keeps feature naming and watched-feature governance centralized.

Function	What it does	Why it matters
_human_feature_name	Converts technical feature names into readable labels.	Improves operator readability.
_build_message	Creates target-specific recommendation text.	Connects ML explanation to MDF process action language.
build_shap_recommendations	Selects top SHAP items and watched features.	Provides concise operational guidance.

9. Risk Evaluation

service/risk_level.py converts numeric predictions into operational risk levels: LOW, MEDIUM, HIGH. This is not ML training; it is a business interpretation layer above ML output.

The method is rule-based thresholding. Some targets use high_is_risky, meaning larger values increase risk. Other targets use low_is_risky, meaning smaller values increase risk. This makes dashboard interpretation easier.

10. Feature Metadata

service/feature_metadata.py loads important features by target for API and GUI summaries. It first tries metadata fields such as important_features, top_features, selected_features, or forced_production_features. If not available, it searches CSV files such as shap_feature_importance.csv or feature_importance.csv. If neither exists, it falls back to the saved features.json order.

Why this matters: the GUI can show important feature context even if a specific explainability report format changes between training runs.

11. API Layer

The API layer exposes runtime functionality through FastAPI.

API file / endpoint	Functionality	Why it exists
api/models.py - /models	Lists artifact directories, registered targets, and registry summary.	Operational visibility.
api/predict.py - /predict	Single-target prediction using single model or ensemble.	Main prediction endpoint.
api/predict.py - /explain	Single-target prediction plus SHAP explanation.	Debugging and operator trust.
api/predict_all.py - /predict-all	Predicts all registered targets and logs each result.	GUI dashboard and production cockpit integration.

12. Schema Layer

schemas/request_schema.py defines input payloads. schemas/response_schema.py defines output contracts. These schemas make Swagger documentation clearer and ensure frontend/GUI integration receives predictable fields.

Important response fields include prediction, risk_level, serving_type, missing_features, extra_features, invalid_numeric_features, null_features, forced_missing_features, explanation, recommendations, ensemble_members,

ensemble_weights, member_predictions, and duration_ms.

13. Database Layer and Traceability

The db layer persists prediction logs into PostgreSQL. This is essential for industrial traceability.

File / Class / Function	Functionality	Why it matters
database.py	Creates SQLAlchemy engine, SessionLocal, Base.	Central DB setup.
dependencies.py.get_db	Provides FastAPI DB session dependency.	Safe session lifecycle.
models.PredictionLog	Defines prediction_log table.	Traceability schema.
prediction_log_repository.save_prediction_log	Writes prediction event to DB and rolls back on failure.	Audit history and GUI monitoring.

PredictionLog stores target, prediction, model, algorithm, serving type, risk level, feature diagnostics, request source, production order, board ID, status, and error message. This supports future drift monitoring, dashboard filters, and production root-cause analysis.

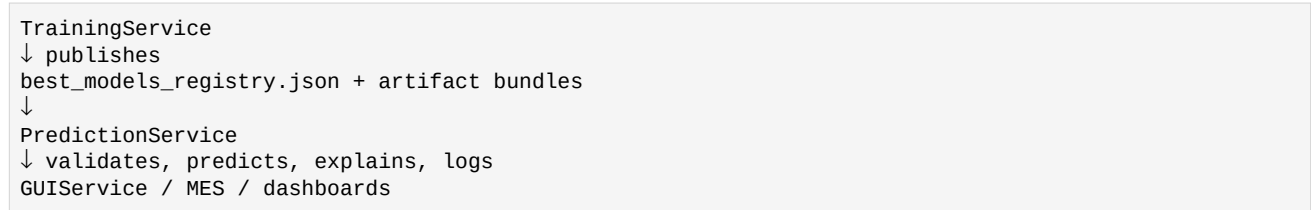
14. main.py Application Lifecycle

main.py defines the FastAPI app lifecycle. On startup it creates/checks database tables, synchronizes artifacts from TrainingService, loads the registry, preloads model artifacts, and starts serving APIs. On shutdown it clears the loaded registry.

The HTTP middleware logs each request method, path, status code, and duration. This makes runtime behavior visible during Swagger testing, GUI integration, and production monitoring.

15. Final Architectural Position

PredictionService is now an industrial online ML serving platform, not just a simple inference script. It is aligned with TrainingService artifact governance and supports registry-driven serving, exact feature-order enforcement, single-model and weighted ensemble prediction, SHAP explainability, recommendation generation, risk interpretation, and PostgreSQL traceability.



Current principle: TrainingService owns model selection and artifact production. PredictionService owns safe runtime enforcement and serving. GUIService owns visualization and decision support.